Wine E-Label Platform — Engineering Audit & Fix Report

Verification, Architectural Solutions, Database Relationships, Internationalization & EU Compliance Implementation

Repository	https://github.com/pranavbhandare31/Wine_lable
Target Branch	main
Technologies	Next.js 14, TypeScript, Tailwind CSS, next-intl, Drizzle ORM, PGlite / PostgreSQL, Clerk Auth
Compliance Standards	EU Regulation 2021/2117 & EU Regulation 1169/2011 (Wine E-Labels & Multilingual Disclosures)
Generated	October 17, 2026

1. Executive Task Summary

Task ID	Description	Status	Resolution Summary
Task 1	Verify and Fix QR Routing Mismatch	**COMPLETE**	Fixed /public/products/:id to singular /public/product/:id (Commit 8ef8138).
Task 2	Audit & Fix createdBy DB Column Type	**COMPLETE**	Altered integer to text in Drizzle & Zod with Migration 0002 for Clerk IDs (Commit 142841c).
Task 3	Product-Ingredient Relationship Solution	**COMPLETE**	Implemented product_ingredients junction table, relations, migration 0003, API, UI, and 7 integration tests.
Task 4	English/French Translation Parity & Fix	**COMPLETE**	Added 'product' namespace to en.json & fr.json, updated public product page to use next-intl keys without fallbacks, added 6 tests (20/20 passing), verified build.

2. Tasks 1 & 2 Overview

Task 1 (QR Route Mismatch): Resolved critical 404 scan error by replacing /public/products/:id with /public/product/:id in qr/page.tsx. Preserved print/download handlers. Pushed in commit 8ef8138.

Task 2 (createdBy DB Type): Updated Schema.ts from integer to text('created_by') and updated Zod validation to accept Clerk string user IDs (e.g. user_2bFvj39...). Created Migration 0002_wild_jane_foster.sql. Pushed in commit 142841c.

3. Task 3: Product–Ingredient Relationship Solution (Deep Dive)

Issue & Architecture: Products and ingredients were isolated without foreign keys. Implemented Many-to-Many junction table product_ingredients with ON DELETE CASCADE, Drizzle relations, migration 0003_thick_dark_beast.sql, UI checkboxes, and public label display.

```
RELATIONAL DATABASE SCHEMA DESIGN
[ products ] (id, name, vintage, wine_type, created_by)
■ 1 ■■■■▼ N [ product_ingredients ] (product_id [FK], ingredient_id [FK]) ▲ N ■■■■ 1 [ ingredients ]
```

4. Task 4: English & French Internationalization Audit & Resolution (Deep Dive)

Issue & Root Cause: Under EU Regulation 2021/2117 and EU 1169/2011, wine e-labels must display mandatory information in the language of the Member State of sale. In src/app/[locale]/public/product/[id]/page.tsx, useTranslations('product') was called, but neither en.json nor fr.json contained the 'product' namespace. Hard-coded fallbacks concealed the error in English, but French consumers visiting /fr/public/product/:id saw completely English text.

Solution Implemented:
1. Added matching 26-key 'product' dictionaries to both src/locales/en.json and src/locales/fr.json with exact French translations (e.g. Millésime, Type de vin, Ingrédients et allergènes, Informations nutritionnelles, Biologique).
2. Updated page.tsx to call unstable_setRequestLocale(params.locale) and removed all hard-coded fallbacks (|| '...').
3. Added status badges, ingredient titles, allergen prefixes, empty states, and nutrition labels bound to t(...).

Key	English (en.json)	French (fr.json)
vintage	Vintage	Millésime
wineType	Type	Type de vin
ingredients_allergens_title	Ingredients & Allergens	Ingrédients et allergènes
allergens_prefix	Allergens	Allergènes
nutritional_info_title	Nutritional Information	Informations nutritionnelles
organic / vegan	Organic / Vegan	Biologique / Végétalien

Automated Verification Summary:

• **i18n Test Suite (src/locales/Translations.test.ts):** 6 tests covering schema, key parity, distinct French values, and next-intl createTranslator execution passed.
• **Full Project Vitest Suite:** 5 test files, 20/20 passed in 5.39s.
• **Production Build:** npm run build passed with exit code 0, generating all static and dynamic routes.

5. Viva Preparation & Explanation for Sir (Tasks 1–4)

Spoken Script for Sir:
"Good morning Sir. In this phase of the Wine E-Label platform, we tackled the internationalization requirements under EU Regulation 2021/2117. We found that the public wine page called useTranslations('product'), but neither en.json nor fr.json had the 'product' namespace defined. Hardcoded fallbacks concealed the issue in English, causing French visitors on /fr/ to see English text. We implemented complete 26-key matching dictionaries for English and French, eliminated fallbacks so missing translations cannot be hidden, and verified everything with 6 automated tests. All 20 tests in the project and the Next.js production build succeeded with 100% confidence."

Key Viva Questions:

Q1: How does next-intl handle translation loading?
Answer: next-intl reads the [locale] URL parameter via middleware. getRequestConfig in src/libs/i18n.ts imports the matching locale JSON file, and useTranslations('namespace') scopes access to the corresponding keys.

Q2: Why avoid hardcoded fallbacks like `t('key') || 'Fallback'`?
Answer: Hardcoded fallbacks silently mask missing translations and prevent non-English users from seeing their language. Removing fallbacks ensures translation keys are verified and properly populated in all language dictionaries.

Q3: Why is multilingual compliance critical under EU 2021/2117?
Answer: EU law strictly requires wine e-labels to be accessible in an official language easily understood in each Member State where the product is sold. Multilingual support is a legal requirement for market entry.